

1 Port Expander LCD

Para reduzir o número de ligações necessárias entre o módulo de controlo e o LCD, foi também utilizado o módulo Port Expander LCD (PELCD), responsável por receber informação em série e convertê-la nos sinais paralelos exigidos pelo display. Esta abordagem permite separar os detalhes de baixo nível do hardware da lógica de controlo implementada em software.

1.1 Port Expander LCD (PELCD)

O módulo Port Expander LCD (PELCD) implementa a interface entre o software de controlo e o display LCD. De acordo com o enunciado do projeto, este módulo recebe a informação enviada em série pelo módulo Control e converte-a nos sinais paralelos necessários ao LCD, nomeadamente o sinal RS, os oito bits de dados e o sinal E. A utilização deste bloco justifica-se pela necessidade de reduzir o número de interligações físicas entre o PC e o hardware. Em vez de o software controlar diretamente todos os sinais paralelos do LCD, envia apenas uma trama série através de três linhas principais:

- LCDsel– sinal de seleção do periférico;
- SCLK– relógio da comunicação série;
- SDX– linha de dados série.

Segundo o diagrama do PELCD e o respetivo protocolo de comunicação, a mensagem enviada para este módulo é constituída por 10 bits de informação :

- Bit RS (bit 0): primeiro bit a ser enviado; indica se a mensagem é de controlo ou de dados para o LCD.
- Bits D0 a D7 (bits 1 a 8): oito bits de dados a entregar ao barramento de dados do LCD.
- Bit E (bit 9): último bit a ser recebido; corresponde ao sinal de enable responsável por desencadear o registo da informação no LCD.

No final, a linha SS efectua uma transição ascendente, sinalizando o fim da comunicação. Este evento é aproveitado internamente para capturar a palavra completa de 10 bits e disponibilizá-la de forma estável na saída paralela do módulo.

O início de cada transmissão é identificado por uma transição descendente em LCDsel, designada por condição de start. Depois disso, os bits são amostrados nas transições ascendentes do sinal

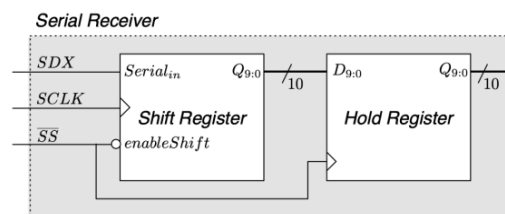


Figura 1: Diagrama de blocos do módulo *SerialReceiver*

SCLK. Internamente, o PELCD contém um bloco Serial Receiver, composto por um Shift Register e por um Hold Register, que recebem e mantêm a trama até esta ser disponibilizada ao LCD. Do ponto de vista do software, este funcionamento é explorado pelo módulo SerialEmitter, que envia bit a bit a trama necessária, e pelo módulo LCD, que organiza os comandos e dados segundo o protocolo esperado. Assim, o PELCD funciona como um elemento intermédio entre a comunicação série gerada em software e a interface paralela exigida pelo display físico.

1.2 SerialReceiver

Foi implementado o bloco Serial Receiver conforme o diagrama de blocos da Figura 2. Este módulo é o núcleo funcional do sistema: coordena a recepção dos dados série e a sua conversão para um formato paralelo de 10 bits, para ser entregue ao LCD. A sua arquitectura é composta pela instanciación e interligação de dois sub-blocos: o ShiftRegister e o HoldRegister10bits. A comunicação entre eles é feita através de um barramento interno de 10 bits que transporta a palavra em construção durante a recepção da trama. Um aspecto central da lógica deste módulo é a geração do sinal de habilitação do deslocamento, designado enableShift no diagrama. Este sinal é obtido por inversão do sinal SS: quando a linha SS está em nível baixo (trama activa), o sinal invertido fica em nível alto, habilitando o ShiftRegister a receber e deslocar os bits à chegada de cada flanco ascendente de SCLK. Quando a mensagem termina e SS sobe, o deslocamento automaticamente acaba. A captura da palavra completa é feita de forma engenhosa: o sinal SS é ligado directamente ao relógio do Hol-

dRegister. Assim, o flanco ascendente de SS, que marca o fim da mensagem, atua simultaneamente como pulso de relógio que captura a palavra de 10 bits presente na saída do ShiftRegister e a transfere para o HoldRegister, onde permanece estável até à próxima mensagem.

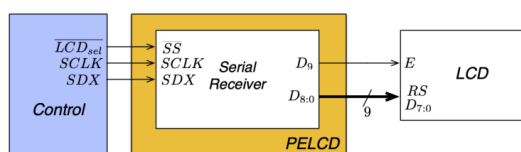


Figura 2: Diagrama de blocos do módulo Port Expander LCD (PELCD)

O módulo PELCD recebe em série uma mensagem constituída por dez (10) bits de informação. A comunicação com este módulo realiza-se segundo o protocolo ilustrado na Figura, em que o bit RS é o primeiro bit de informação e indica se a mensagem é de controlo ou dados. Os seguintes oito (8) bits contêm os dados a entregar ao LCD. O último bit contém o sinal de E responsável por registar a informação no LCD.

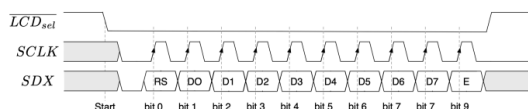


Figura 3: Protocolo de comunicação com o módulo Port Expander LCD (PELCD)

O emissor, realizado em software, quando pretende enviar uma trama para o módulo PELCD promove uma condição de início de trama (Start), que corresponde a uma transição descendente na linha (LCDsel). Após a condição de início, o módulo PELCD armazena os bits de dados da trama nas transições ascendentes do sinal SCLK. O bloco Serial Receiver do módulo PELCD é constituído por dois blocos principais: i) um registo de deslocamento; e ii) um registo de manutenção; designados por Shift Register e Hold Register respetivamente.

1.3 Shift Register

O registo de deslocamento de 10 bits é responsável por receber os dados do protocolo série e convertê-los numa palavra paralela. Internamente, o módulo é constituído por dez flip-flops D (FFD): a entrada série SDX alimenta o primeiro flip-flop, e a saída de cada flip-flop é ligada à entrada do seguinte, formando uma cadeia de deslocamento. Todos os flip-flops são sincronizados pelo sinal de relógio SCLK e partilham o mesmo sinal de habilitação enableShift (EN_shift), proveniente do SerialReceiver. Enquanto este sinal estiver activo, cada flanco ascendente de SCLK provoca o avanço de um bit na cadeia: o bit mais recente entra pelo primeiro flip-flop e todos os anteriores são deslocados uma posição.

Após dez ciclos de SCLK, a cadeia contém a totalidade dos 10 bits da mensagem. No entanto, o bit que entrou em primeiro lugar encontra-se no último flip-flop da cadeia. Para garantir que a saída paralela apresenta os bits na ordem correcta do protocolo, RS na posição 0, D0 na posição 1, e assim sucessivamente até E na posição 9, as saídas dos flip-flops são ligadas à saída do módulo em ordem inversa à da sua posição na cadeia.

1.4 Hold Register 10bits

O registo de manutenção de 10 bits, que papel é capturar a palavra paralela produzida pelo ShiftRegister no momento exacto em que a trama termina, e mantê-la estável na saída durante o intervalo entre mensagens. Ao contrário do ShiftRegister, em que os flip-flops estão encadeados, o HoldRegister é composto por dez flip-flops D completamente independentes entre si, cada um responsável por armazenar individualmente um bit do armazenamento de 10 bits. Não existe qualquer ligação entre flip-flops: cada um recebe directamente o bit correspondente da saída do ShiftRegister e captura-o no flanco ascendente do seu relógio. O relógio deste registo é o próprio sinal SS, conforme ligado pelo SerialReceiver. Desta forma, a ascendencia de SS, que ocorre no fim de cada mensagem, é o evento que provoca a captura simultânea dos 10 bits. A partir desse instante, a saída D[9:0] fica estável e disponível para o LCD consumir, mesmo que uma nova mensagem já esteja a ser recebida no ShiftRegister.

2 Conclusões

O módulo PELCD assume um papel central na experiência do utilizador. O LCD é o principal meio de comunicação entre a máquina e quem a opera, sendo através dele que são apresentadas informações essenciais como o tipo de bilhete, o preço, o troco a devolver ou mensagens de estado do sistema. Sem um módulo de interface fiável e correcto entre o controlador e o LCD, toda a lógica de controlo da máquina ficaria sem forma de comunicar os seus resultados ao exterior, tornando o sistema inutilizável na prática.

A VHDL

A.1 Port Expander LCD

```
library ieee;
use ieee.std_logic_1164.all;

entity PELCD is
    port(
        SS    : in std_logic;
        SCLK  : in std_logic;
        SDX    : in std_logic;
        Rst    : in std_logic;
        D      : out std_logic_vector(9 downto 0)
    );
end PELCD;

architecture structural of PELCD is

    component SerialReceiver is
        port(
            SDX    : in std_logic;
            SCLK    : in std_logic;
            SS      : in std_logic;
            Rst     : in std_logic;
            D       : out std_logic_vector(9 downto 0)
        );
    end component;

begin

    SerialReceiver_V: serialReceiver
        port map(
            SDX => SDX,
            SCLK => SCLK,
            SS  => SS,
            Rst => Rst,
            D   => D
        );
end structural;
```



instituto
superior de
engenharia
de lisboa

Keyboard Reader (*Ticket Machine*)

Laboratório de Informática e Computadores 2025 / 2026 verão

Autores: David Velez / Nuno Sebastião / Pedro Miguens / Rogério Rebelo / Rui Duarte

A.2 Serial Receiver

```
library ieee;
use ieee.std_logic_1164.all;

entity SerialReceiver is
    port(
        SDX      : in std_logic;
        SCLK      : in std_logic;
        SS        : in std_logic;
        Rst        : in std_logic;
        D          : out std_logic_vector(9 downto 0)
    );
end SerialReceiver;

architecture structural of SerialReceiver is

    component ShiftRegister is
        port(
            Serial_in : in std_logic;
            EN_shift   : in std_logic;
            SCLK        : in std_logic;
            Rst          : in std_logic;
            Q_OUT        : out std_logic_vector(9 downto 0)
        );
    end component;

    component HoldRegister10bits is
        port(
            CLK      : in STD_LOGIC;
            Rst       : in STD_LOGIC;
            SET       : in STD_LOGIC;
            EN        : in STD_LOGIC;
            D_IN      : in STD_LOGIC_VECTOR(9 downto 0);
            Q_OUT     : out STD_LOGIC_VECTOR(9 downto 0)
        );
    end component;

    signal s_D_in      : std_logic_vector(9 downto 0);
    signal s_not_ss     : std_logic;

begin

    ShiftRegister_V : ShiftRegister
        port map(
            Serial_in => SDX,
            EN_shift   => s_not_ss,
            SCLK        => SCLK,
            Rst          => Rst,
            Q_OUT        => s_D_in
        );

    HoldRegister_V : HoldRegister10bits
        port map(
            CLK      => SS,
            Rst       => Rst,
            SET       => '0',
            EN        => '1',
            D_IN      => s_D_in,
            Q_OUT     => D
        );

    s_not_ss <= not SS;

end structural;
```



instituto
superior de
engenharia
de lisboa

Keyboard Reader (*Ticket Machine*)

Laboratório de Informática e Computadores 2025 / 2026 verão

Autores: David Velez / Nuno Sebastião / Pedro Miguens / Rogério Rebelo / Rui Duarte

A.3 Shift Register

```

library ieee;
use ieee.std_logic_1164.all;

entity ShiftRegister is
    port(
        Serial_in : in std_logic;
        EN_shift   : in std_logic;
        SCLK       : in std_logic;
        Rst        : in std_logic;
        Q_OUT      : out std_logic_vector(9 downto 0)
    );

end ShiftRegister;

architecture structural of ShiftRegister is

    component FFD is
        port(
            CLK      : in  STD_LOGIC;
            Rst      : in  STD_LOGIC;
            SET      : in  STD_LOGIC;
            D        : in  STD_LOGIC;
            EN       : in  STD_LOGIC;
            Q        : out STD_LOGIC
        );
    end component;

    signal Q_s : std_logic_vector(9 downto 0);

begin

    FFO : FFD
        PORT MAP (
            CLK  => SCLK,
            Rst  => Rst,
            SET  => '0',
            EN   => EN_shift,
            D    => Serial_in,
            Q    => Q_s(0)
        );

    FF1 : FFD
        PORT MAP (
            CLK  => SCLK,
            Rst  => Rst,
            SET  => '0',
            EN   => EN_shift,
            D    => Q_s(0),
            Q    => Q_s(1)
        );

    FF2 : FFD
        PORT MAP (
            CLK  => SCLK,
            Rst  => Rst,
            SET  => '0',
            EN   => EN_shift,
            D    => Q_s(1),
            Q    => Q_s(2)
        );

    FF3 : FFD
        PORT MAP (
            CLK  => SCLK,
            Rst  => Rst,
            SET  => '0',
            EN   => EN_shift,
            D    => Q_s(2)
        );

```



instituto
superior de
engenharia
de lisboa

Keyboard Reader (*Ticket Machine*)

Laboratório de Informática e Computadores 2025 / 2026 verão

Autores: David Velez / Nuno Sebastião / Pedro Miguens / Rogério Rebelo / Rui Duarte

A.4 Hold Register 10bits

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY HoldRegister10bits IS
    PORT(
        CLK      : in  STD_LOGIC;
        Rst      : in  STD_LOGIC;
        SET      : in  STD_LOGIC;
        EN       : in  STD_LOGIC;
        D_IN     : in  STD_LOGIC_VECTOR(9 downto 0);
        Q_OUT    : out STD_LOGIC_VECTOR(9 downto 0)
    );
END HoldRegister10bits;

ARCHITECTURE structural OF HoldRegister10bits IS

    COMPONENT FFD IS
        PORT(
            CLK      : in  STD_LOGIC;
            Rst      : in  STD_LOGIC;
            SET      : in  STD_LOGIC;
            D        : in  STD_LOGIC;
            EN       : in  STD_LOGIC;
            Q        : out STD_LOGIC
        );
    END COMPONENT;

BEGIN

    FF0 : FFD
        PORT MAP (
            CLK => CLK,
            Rst => Rst,
            SET => '0',
            EN  => '1',
            D   => D_IN(0),
            Q   => Q_OUT(0)
        );

    FF1 : FFD
        PORT MAP (
            CLK => CLK,
            Rst => Rst,
            SET => '0',
            EN  => '1',
            D   => D_IN(1),
            Q   => Q_OUT(1)
        );

    FF2 : FFD
        PORT MAP (
            CLK => CLK,
            Rst => Rst,
            SET => '0',
            EN  => '1',
            D   => D_IN(2),
            Q   => Q_OUT(2)
        );

    FF3 : FFD
        PORT MAP (
            CLK => CLK,
            Rst => Rst,
            SET => '0',
            EN  => '1',
            D   => D_IN(3),
            Q   => Q_OUT(3)
        );

    FF4 : FFD
        PORT MAP (

```

B *Kotlin*

B.1 TUI

Algoritmo 1: TUI - Hardware Abstract Layer

```
object TUI {

    val Column = 16
    var Text = ""

    fun init() {
        KBD.init()
        LCD.init()
        LCD.clear()
    }

    fun run() {
        while (true) {
            val key = KBD.getKey()

            if (key != KBD.none) {
                if (key == '#') {
                    Text = ""
                } else if (key == 'C') {
                    Text += "Bilhete para o carralho?"
                } else {
                    Text += key
                }

                LCD.clear()
                val lengthText = Text.length

                if (lengthText <= Column) {
                    LCD.cursor(0, 0)
                    LCD.write(Text)
                }

                else {

                    val upText = Text.substring(0, Column)
                    LCD.cursor(0, 0)
                    LCD.write(upText)

                    val lowText = Text.substring(Column)

                    val emptySpace = Column - lowText.length
                    val firstColumn = emptySpace / 2

                    LCD.cursor(1, firstColumn)
                    LCD.write(lowText)
                }
            }
        }
    }

    fun main() {
        TUI.init()
        TUI.run()
    }
}
```

B.2 Serial Emitter

Algoritmo 2: SerialEmmitter

```
import isel.leic.utils.Time

object SerialEmitter {
    enum class Peripheral { LCD, TICKET }
}
```

```

private const val SDX_MASK = 0x01      // 00
private const val SCLK_MASK = 0x02     // 01
private const val LCDSEL_MASK = 0x04   // 02
private const val TICKETSEL_MASK = 0x08 // 03

fun init() {
    HAL.init()
    HAL.clrBits(SDX_MASK)
    HAL.clrBits(SCLK_MASK)
    HAL.setBits(LCDSEL_MASK)
    HAL.setBits(TICKETSEL_MASK)
}

fun send(addr: Peripheral, data: Int) {
    when (addr) {
        Peripheral.LCD -> sendLCD(data)
        Peripheral.TICKET -> sendTicket(data)
    }
}

private fun sendLCD(data: Int) {
    HAL.setBits(LCDSEL_MASK)
    HAL.clrBits(SCLK_MASK)

    HAL.clrBits(LCDSEL_MASK)

    for (i in 0..9) {
        HAL.clrBits(SCLK_MASK)

        val bit = (data shr i) and 1
        if (bit == 1) HAL.setBits(SDX_MASK)
        else HAL.clrBits(SDX_MASK)

        Time.sleep(1)
        HAL.setBits(SCLK_MASK)
        Time.sleep(1)
    }

    HAL.clrBits(SCLK_MASK)
    HAL.setBits(LCDSEL_MASK)
}

private fun sendTicket(data: Int) {
    HAL.setBits(TICKETSEL_MASK)
    HAL.clrBits(SCLK_MASK)

    HAL.clrBits(TICKETSEL_MASK)

    for (i in 0..9) {
        HAL.clrBits(SCLK_MASK)

        val bit = (data shr i) and 1
        println("clock_${i+1}_->bit=${bit}")

        if (bit == 1) HAL.setBits(SDX_MASK)
        else HAL.clrBits(SDX_MASK)

        Time.sleep(1)
        HAL.setBits(SCLK_MASK)
        Time.sleep(1)
    }

    HAL.clrBits(SCLK_MASK)
    HAL.setBits(TICKETSEL_MASK)
}

fun isBusy(): Boolean = false
}

```

B.3 LCD

Algoritmo 3: LCD - Hardware Abstract Layer

```

import isel.leic.utils.Time

object LCD {
  const val LINES = 2
  const val COLS = 16

  private const val FUNCTION_SET = 0x38
  private const val DISPLAY_ON = 0x0C
  private const val ENTRY_MODE = 0x06
  private const val CLEAR_DISPLAY = 0x01
  private const val SET_DDGRAM = 0x80

  private fun writeByteSerial(rs: Boolean, data: Int) {
    val rsBit = if (rs) 1 else 0

    val frameEnable = (1 shl 9) or ((data and 0xFF) shl 1) or rsBit
    val frameDisable = ((data and 0xFF) shl 1) or rsBit

    SerialEmitter.send(SerialEmitter.Peripheral.LCD, frameEnable)
    Time.sleep(1)
    SerialEmitter.send(SerialEmitter.Peripheral.LCD, frameDisable)
    Time.sleep(1)
  }

  private fun writeByte(rs: Boolean, data: Int) {
    writeByteSerial(rs, data)
  }

  private fun writeCMD(data: Int) {
    writeByte(false, data)
  }

  private fun writeDATA(data: Int) {
    writeByte(true, data)
  }

  fun init() {
    SerialEmitter.init()

    Time.sleep(50)
    writeCMD(0x30)
    Time.sleep(5)
    writeCMD(0x30)
    Time.sleep(1)
    writeCMD(0x30)
    Time.sleep(1)

    writeCMD(FUNCTION_SET)
    writeCMD(DISPLAY_ON)
    writeCMD(ENTRY_MODE)
    clear()
  }

  fun write(c: Char) {
    writeDATA(c.code)
  }

  fun write(text: String) {
    for (c in text) write(c)
  }

  fun cursor(line: Int, column: Int) {
    if (line !in 0 until LINES) return
    if (column !in 0 until COLS) return

    val address = if (line == 0) column else 0x40 + column
    writeCMD(SET_DDGRAM or address)
  }

  fun clear() {
    writeCMD(CLEAR_DISPLAY)
    Time.sleep(2)
  }
}

```

```

    }

    private const val SET_CGRAM = 0x40

    fun createChar(location: Int, pattern: IntArray) {
        val loc = location and 0x7 // s 0..7

        writeCMD(SET_CGRAM or (loc shl 3))

        for (i in 0 until 8) {
            writeDATA(pattern[i] and 0x1F)
        }
    }

    fun writeCharCode(code: Int) {
        writeDATA(code and 0xFF)
    }

    fun loadEuroSymbol(location: Int = 0) {
        val euro = intArrayOf(
            0b01110,
            0b10001,
            0b11100,
            0b10000,
            0b11100,
            0b10001,
            0b01110,
            0b00000
        )
        createChar(location, euro)
    }
}

fun main() {
    LCD.init()
    LCD.loadEuroSymbol(0)
    LCD.cursor(0, 0)
    LCD.write("01a")
    LCD.cursor(1, 0)
    LCD.write("4,00")
    LCD.writeCharCode(0)
}

```

B.4 Serial Receiver

Algoritmo 4: SerialReceiver

```

import isel.leic.utils.Time

const val INVALID_KEYCODE = -1

object SerialReceiver {

    private const val RXD_MASK = 0x80 // I7
    private const val RXCLK_MASK = 0x80 // O7

    private const val HALF_CLOCK_MS = 2L
    private const val SAMPLE_DELAY_MS = 2L
    private const val FRAME_END_DELAY_MS = 2L

    fun init() {
        HAL.clrBits(RXCLK_MASK)
    }

    fun rcv(): Int {
        // sem trama
        if (HAL.isBit(RXD_MASK)) return INVALID_KEYCODE

        // pequena espera para garantir start est vel
        Time.sleep(SAMPLE_DELAY_MS)

        // clock 1: deve dar 1
        pulseClock()
    }
}

```

```

Time.sleep(SAMPLE_DELAY_MS)
if (!HAL.isBit(RXD_MASK)) return INVALID_KEYCODE

// clocks 2..5: K0..K3
var keyCode = 0
for (i in 0..3) {
    pulseClock()
    Time.sleep(SAMPLE_DELAY_MS)

    if (HAL.isBit(RXD_MASK)) {
        keyCode = keyCode or (1 shl i)
    }
}

// clock 6: deve dar 0
pulseClock()
Time.sleep(SAMPLE_DELAY_MS)
if (HAL.isBit(RXD_MASK)) return INVALID_KEYCODE

// clock 7: deve dar 1
pulseClock()
Time.sleep(SAMPLE_DELAY_MS)
if (!HAL.isBit(RXD_MASK)) return INVALID_KEYCODE

// clock extra para fechar a trama
pulseClock()
Time.sleep(FRAME_END_DELAY_MS)

// garantir que voltou a idle
if (!HAL.isBit(RXD_MASK)) return INVALID_KEYCODE

return keyCode
}

fun waitRcv(): Int {
    while (HAL.isBit(RXD_MASK)) {
        Time.sleep(1)
    }
    return rcv()
}

private fun pulseClock() {
    HAL.setBits(RXCLK_MASK)
    Time.sleep(HALF_CLOCK_MS)
    HAL.clrBits(RXCLK_MASK)
    Time.sleep(HALF_CLOCK_MS)
}
}

```